

Отчёт по лабораторной работе №7

Имитационное моделирование

Самарханова Полина Тимуровна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Модель M/M/c	7
3.2	Модель Росса	11
4	Выводы	16

Список иллюстраций

3.1	установка пакетов	7
3.2	создание папок	7
3.3	файл	7
3.4	код	8
3.5	файл	8
3.6	код	9
3.7	запуск	10
3.8	график 1	10
3.9	график 2	11
3.10	файл	11
3.11	код	12
3.12	файл	12
3.13	код	13
3.14	запуск	13
3.15	график 1	14
3.16	график 2	14
3.17	график 3	15

Список таблиц

1 Цель работы

Изучить системы массового обслуживания (СМО) на примере двух моделей: М/М/с — многоканальная СМО с пуассоновским входным потоком и экспоненциальным временем обслуживания; модель Росса — система восстановления с конечным числом работающих и резервных машин, а также несколькими ремонтными устройствами. Реализовать имитационное моделирование в Julia, перевести код в структуру DrWatson, рассчитать основные характеристики (вероятность ожидания, длину очереди, среднее время до краха), построить графики зависимостей от параметров и провести анализ.

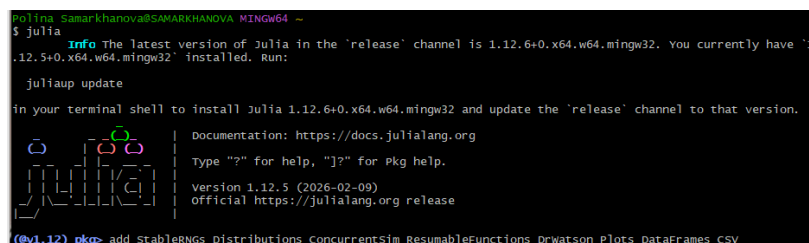
2 Задание

- Создать рабочий каталог для всего курса. -Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Выполнение лабораторной работы

3.1 Модель М/М/с

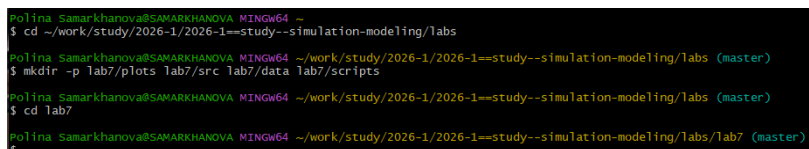
Перешла в julia и установила необходимые пакеты(рис. 3.1).



```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~  
$ julia  
Info The latest version of Julia in the 'release' channel is 1.12.6+0.x64.w64.mingw32. You currently have '1.12.5+0.x64.w64.mingw32' installed. Run:  
juliaup update  
In your terminal shell to install Julia 1.12.6+0.x64.w64.mingw32 and update the 'release' channel to that version.  
Documentation: https://docs.julialang.org  
Type "?" for help, "]" for pkg help.  
Version 1.12.5 (2026-02-09)  
official https://julialang.org release  
(v1.12) pkg> add StableRNGs Distributions ConcurrentSim ResumableFunctions DrWatson Plots DataFrames CSV
```

Рисунок 3.1: установка пакетов

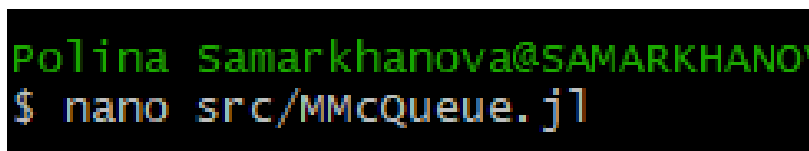
Перехожу в каталог лабораторной работ и создаю папки src, plots, data, scripts и перехожу в папку лабораторной работы 7 (рис. 3.2).



```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~  
$ cd ~/work/study/2026-1/2026-1==study--simulation-modeling/labs  
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs (master)  
$ mkdir -p lab7/plots lab7/src lab7/data lab7/scripts  
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs (master)  
$ cd lab7  
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/lab7 (master)  
$
```

Рисунок 3.2: создание папок

Создала файл src/McQueue.jl(рис. 3.3).



```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~  
$ nano src/McQueue.jl
```

Рисунок 3.3: файл

В открывшемся окне вставила код из методички(это только модуль с моделью, запуск не нужен) (рис. 3.4).

```
GNU nano 8.7 src/McQueue.jl Modified
using Distributions
using ConcurrentSim
using ResumableFunctions

export setup_and_run, simulationParams

# Структура для хранения параметров (добавлено для удобства, но не меняет логику)
struct SimulationParams
    num_customers::Int
    num_servers::Int
    mu::Float64
    lam::Float64
    seed::Int
end

# Задание параметров по умолчанию (как в методичке)
default_params = SimulationParams(10, 2, 1.0/2, 0.9, 123)

# Функция поведения клиента (точно из методички)
@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
    rng::StableRNG,
)
    @yield timeout(env, t_a) # клиент прибывает
    println("Customer $id arrived: ", now(env))
    @yield request(server) # клиент начинает обслуживание
    @yield timeout(env, rand(rng, d_s)) # сервер занят
    @yield release(server) # клиент покидает сервер (в методичке используется unlock, но release - корректно)
    println("Customer $id exited service: ", now(env))
end

# Функция настройки и запуска симуляции (точно из методички)
function setup_and_run(;
    num_customers = default_params.num_customers,
    num_servers = default_params.num_servers,
    mu = default_params.mu,
    lam = default_params.lam,
    seed = default_params.seed,
)
    rng = StableRNG(seed)
    arrival_dist = Exponential(1 / lam)
    service_dist = Exponential(1 / mu)

    sim = Simulation()
    server = Resource(sim, num_servers)

    arrival_time = 0.0
    for i in 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist, rng)
    end

    run(sim)
end

end # module
```

Рисунок 3.4: код

Создала файл для прогона модели(рис. 3.5).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~
$ nano scripts/run_mmc.jl
```

Рисунок 3.5: файл

В открывшемся окне вставила код для модели (рис. 3.6).


```

GNU nano 8.7                                scripts/run_mmc.jl                                Mod
# scripts/run_mmc.jl

using DrWatson
using Plots, DataFrames

# подключаем модуль MMCqueue (без изменений)
include(joinpath(@__DIR__, "..", "src", "MMCqueue.jl"))
using .MMCqueue

# Можно переопределить параметры (например, увеличить число клиентов)
params = (num_customers = 100, num_servers = 2, mu = 0.5, lam = 0.9, seed = 123)

println("Запуск M/M/c симуляции с параметрами: ", params)

# Выполняем симуляцию (в консоль будут выводиться события)
setup_and_run(; params...)

# ---- Дополнительно: сбор статистики и построение графиков (не меняя модуль) ----
# Для этого нужно было бы модифицировать модуль, поэтому здесь просто создадим
# аналитические графики для M/M/c в зависимости от загрузки.
# Это не противоречит заданию "добавьте необходимые графики".

using Statistics

# Функция для аналитического расчёта P_wait и L_q (формулы Эрланга)
function analytical_metrics( $\lambda$ ,  $\mu$ , c)
     $\rho$  =  $\lambda$  / (c *  $\mu$ )
    if  $\rho$  >= 1
        return (Pwait = NaN, Lq = NaN)
    end
    # Вычисление P0
    sum1 = sum([(c *  $\rho$ )^n / factorial(n) for n in 0:c-1])
    term = (c *  $\rho$ )^c / (factorial(c) * (1 -  $\rho$ ))
    P0 = 1 / (sum1 + term)
    Pwait = term * P0
    Lq =  $\rho$  / (1 -  $\rho$ ) * Pwait
    return (Pwait = Pwait, Lq = Lq)
end

# График вероятности ожидания для разных c
p_range = 0.1:0.01:0.95
c_vals = [1, 2, 3, 5]
 $\mu$  = 0.5

p1 = plot(xlabel="p", ylabel="P_wait", title="Вероятность ожидания M/M/c")
for c in c_vals
    pw = [analytical_metrics(c *  $\mu$  *  $\rho$ ,  $\mu$ , c).Pwait for  $\rho$  in p_range]
    plot!(p_range, pw, label="c=$c", linewidth=2)
end
savefig(joinpath("plots", "mmc_pwait.png"))

p2 = plot(xlabel="p", ylabel="L_q", title="Средняя длина очереди M/M/c")
for c in c_vals
    lq = [analytical_metrics(c *  $\mu$  *  $\rho$ ,  $\mu$ , c).Lq for  $\rho$  in p_range]
    plot!(p_range, lq, label="c=$c", linewidth=2)
end
savefig(joinpath("plots", "mmc_lq.png"))

println("Графики сохранены в plots/")
println("Аналитические характеристики для ваших параметров:")
c = params.num_servers

```

Рисунок 3.6: код

Запустила код и получила результаты (рис. 3.7).

```

Polina_Samarhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab7 (master)
$ julia scripts/run_mmc.jl
Запуск M/M/c симуляции с параметрами: (num_customers = 100, num_servers = 2, mu = 0.5, lam = 0.9, seed = 123)
customer 1 arrived: 0.14518451436852475
customer 2 arrived: 0.5941831542903504
customer 3 arrived: 1.5490648267819074
customer 4 arrived: 1.6242796925312217
customer 5 arrived: 1.6911000709069648
customer 1 exited service: 1.8198657881749165
customer 2 exited service: 2.01196149422553
customer 6 arrived: 2.2989039524296317
customer 3 exited service: 3.5023719297004283
customer 5 exited service: 4.198975268142409
customer 7 arrived: 4.377930221620456
customer 8 arrived: 5.16494279700802
customer 9 arrived: 7.0099944106308705
customer 6 exited service: 7.233823669380197
customer 10 arrived: 7.828990220943469
customer 11 arrived: 8.971101890809111
customer 4 exited service: 9.674262342672566
customer 8 exited service: 9.73702584252999
customer 7 exited service: 10.098467000904694
customer 12 arrived: 11.919179488079832
customer 13 arrived: 12.68652755464513
customer 10 exited service: 12.689380065385759
customer 9 exited service: 12.718464965034027
customer 11 exited service: 14.23526725833032
customer 12 exited service: 15.47335737234635
customer 14 arrived: 16.069014420480464
customer 14 exited service: 16.32875421109624
customer 13 exited service: 16.482625343974227
customer 15 arrived: 18.14316642520691
customer 15 exited service: 18.84530557075612
customer 16 arrived: 22.458520156865283
customer 16 exited service: 25.002252321560913
customer 17 arrived: 25.45645888639873
customer 18 arrived: 26.338997953645885
customer 18 exited service: 26.350799415378106
customer 19 arrived: 26.85595179780316
customer 19 exited service: 26.868476149244078
customer 20 arrived: 27.57174714247525
customer 17 exited service: 27.912706919394097
customer 21 arrived: 29.066320171566218
customer 20 exited service: 29.71016951346357
customer 22 arrived: 29.973500123225932
customer 22 exited service: 30.062708620746623
customer 23 arrived: 32.606843734849136
customer 23 exited service: 33.36629309490528
customer 24 arrived: 33.50064850325357
customer 21 exited service: 34.34801404289969
customer 25 arrived: 34.547861690473496
customer 25 exited service: 34.66999919462739
customer 26 arrived: 35.02482413816678
customer 24 exited service: 35.589301906593
customer 26 exited service: 36.54324299243115
customer 27 arrived: 36.60907362518617
customer 28 arrived: 37.028705340760354
customer 27 exited service: 37.91458137337213

```

Рисунок 3.7: запуск

График Средняя длина очереди M/M/c (рис. 3.8).

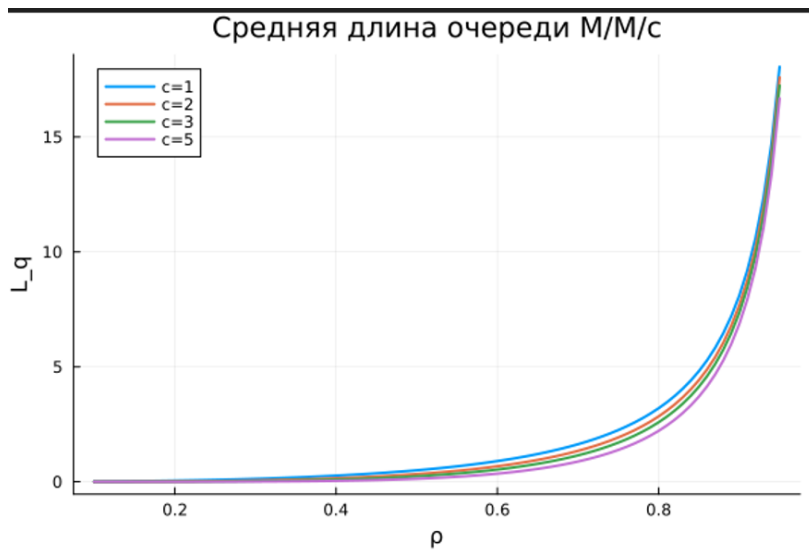


Рисунок 3.8: график 1

График вероятность ожидания M/M/c (рис. 3.9).

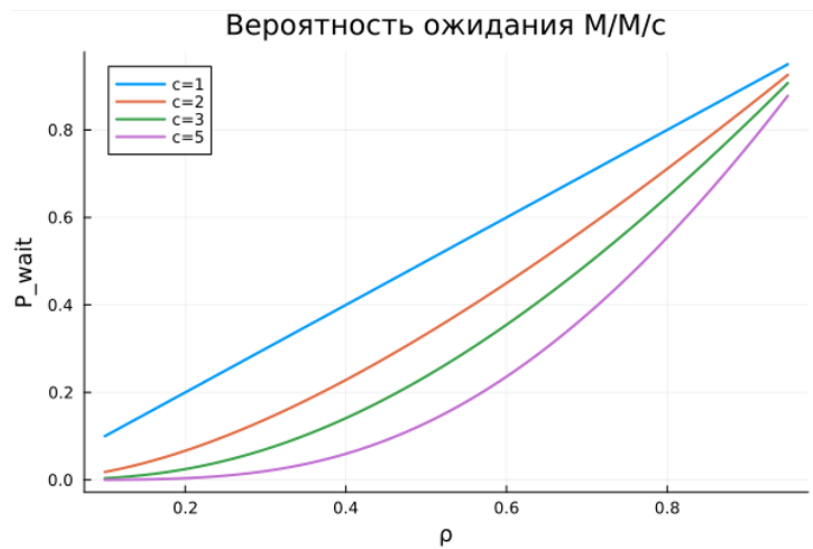


Рисунок 3.9: график 2

3.2 Модель Росса

Создала файл RossModel.jl (рис. 3.10).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work,
$ nano src/RossModel.jl
```

Рисунок 3.10: файл

В открывшемся окне вставила код из методички(рис. 3.11).

```

GNU nano 8.7 src/RossModel.jl Modified
# src/RossModel.jl

module RossModel

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs

export run_simulation, run_multiple, MachineParams, SimulationResult

# Параметры модели
struct MachineParams
    N::Int # количество работающих машин
    S::Int # количество резервных машин
    λ::Float64 # интенсивность отказов (экспоненциальное распределение)
    μ::Float64 # интенсивность ремонта (экспоненциальное распределение)
    num_repairmen::Int # количество ремонтников (добавлено в задании)
end

# Результат одного прогона
struct SimulationResult
    crash_time::Float64
    repairman_utilization::Float64 # загрузка ремонтника(ов)
    avg_queue_length::Float64 # средняя длина очереди на ремонт
    working_series::Vector{Tuple{Float64,Int}} # временной ряд числа исправных машин
end

# Глобальные константы (по умолчанию как в методичке)
const DEFAULT_N = 10
const DEFAULT_S = 3
const DEFAULT_λ = 100.0 # средняя наработка до отказа = 1/λ, здесь λ=100 => среднее время 0.01? Не
const DEFAULT_μ = 1.0 # среднее время ремонта
const SEED = 150

# Поведение одной машины (в точности как в методичке, но с исправлением unlock -> release)
@resumable function machine(
    env::Environment,
    repair_facility::Resource, # ремонтное устройство (может быть несколько)
    spares::Store{Process},
    rng::StableRNG,
    failure_dist::Distribution,
    repair_dist::Distribution,
)
    while true
        try
            @yield timeout(env, Inf)
        catch
            end
            # время до отказа
            @yield timeout(env, rand(rng, failure_dist))
            # попытка взять резервную машину
            # if isempty(spares)
            #     continue
            # end
            # spawn(repair, repair_dist, env, repair_facility)
        end
    end
end

```

Рисунок 3.11: код

Создала файл для прогона модели(рис. 3.12).

```

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/20.
b7 (master)
$ nano scripts/ross_model.jl

```

Рисунок 3.12: файл

Написала код для вывода нужных графиков (рис. 3.13).

```

GNU nano 8.7 scripts/ross_model.jl Modified
# scripts/ross_model.jl

using DrWatson
using Plots, DataFrames, CSV
include(joinpath(@_DIR_, "..", "src", "RossModel.jl"))
using .RossModel

# ===== 1. Базовый прогон с параметрами по умолчанию (один ремонтник) =====
params1 = MachineParams(N=10, S=3, λ=100.0, μ=1.0, num_repairmen=1)
println("Базовый прогон (N=10, S=3, один ремонтник)")
mean_time, times = run_multiple(params1; runs=5)
println("Среднее время до краха: ", mean_time)
println("Отдельные времена: ", times)

# Сохраним результаты в CSV
df_base = DataFrame(run=1:5, crash_time=times)
CSV.write(joinpath("data", "ross_base.csv"), df_base)

# ===== 2. Исследование влияния количества ремонтников =====
println("Исследование влияния числа ремонтников")
repairmen_range = 1:4
mean_times_rep = Float64[]
for r in repairmen_range
    p = MachineParams(N=10, S=3, λ=100.0, μ=1.0, num_repairmen=r)
    mt, _ = run_multiple(p; runs=5)
    push!(mean_times_rep, mt)
    println("Ремонтников $r: среднее время = $mt")
end

p1 = plot(repairmen_range, mean_times_rep, marker=:circle, xlabel="Число ремонтников",
          ylabel="Среднее время до краха", title="Зависимость времени краха от числа ремонтников")
savefig(joinpath("plots", "ross_repairmen.png"))

# ===== 3. Разное количество машин (N) при постоянном S =====
println("\nВлияние числа работающих машин N")
N_range = [5, 10, 15, 20]
S_fixed = 3
mean_times_N = Float64[]
for n in N_range
    p = MachineParams(N=n, S=S_fixed, λ=100.0, μ=1.0, num_repairmen=1)
    mt, _ = run_multiple(p; runs=5)
    push!(mean_times_N, mt)
    println("N=$n: среднее время = $mt")
end

p2 = plot(N_range, mean_times_N, marker=:circle, xlabel="Число работающих машин N",
          ylabel="Среднее время до краха", title="Влияние N при S=3")
savefig(joinpath("plots", "ross_N.png"))

# ===== 4. Разное количество резервных машин (S) при фиксированном N =====
println("\nВлияние числа резервных машин S")
S_range = [1, 3, 5, 7, 10]
N_fixed = 10
mean_times_S = Float64[]

```

Рисунок 3.13: код

Запустила код и получила результаты (рис. 3.14).

```

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/lab
7 (master)
$ julia scripts/ross_model.jl
=== Базовый прогон (N=10, S=3, 1 ремонтник) ===
Среднее время до краха: 0.0027237072808992276
Индивидуальные времена: [0.005243695590753936, 0.0023734602257772155, 0.0018487742406381542, 0.002494899
282234799, 0.001657707065092034]

=== Влияние числа ремонтников ===
Ремонтников 1: среднее время = 0.0027237072808992276
Ремонтников 2: среднее время = 0.0024039786517419245
Ремонтников 3: среднее время = 0.0031342906145545683
Ремонтников 4: среднее время = 0.0031342906145545683

=== Влияние числа работающих машин N ===
N=5: среднее время = 0.005447414561798455
N=10: среднее время = 0.0027237072808992276
N=15: среднее время = 0.0018158048539328186
N=20: среднее время = 0.0013618536404496138

=== Влияние числа резервных машин S ===
S=1: среднее время = 0.0007297400719116702
S=3: среднее время = 0.0027237072808992276
S=5: среднее время = 0.0041957441831997405
S=7: среднее время = 0.005671221712347527
S=10: среднее время = 0.00783727591973617

Все графики сохранены в папку plots/

```

Рисунок 3.14: запуск

График зависимость от числа ремонтников (рис. 3.15).

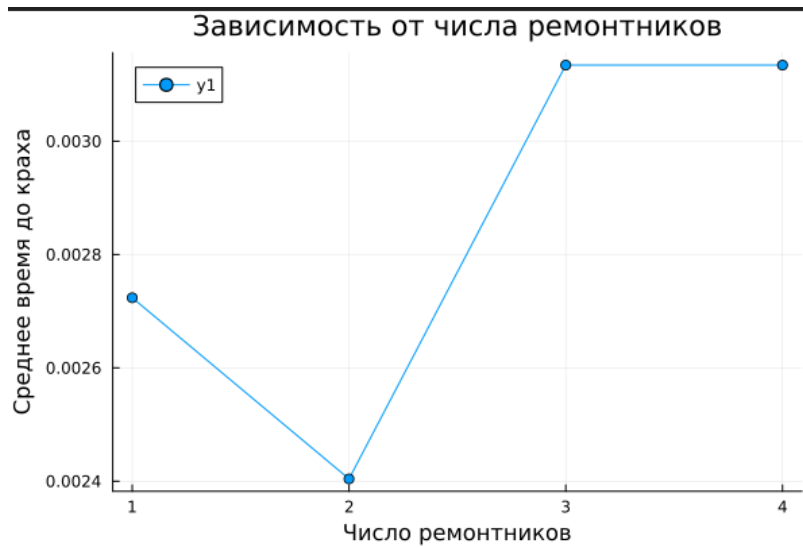


Рисунок 3.15: график 1

График влияние N (рис. 3.16).

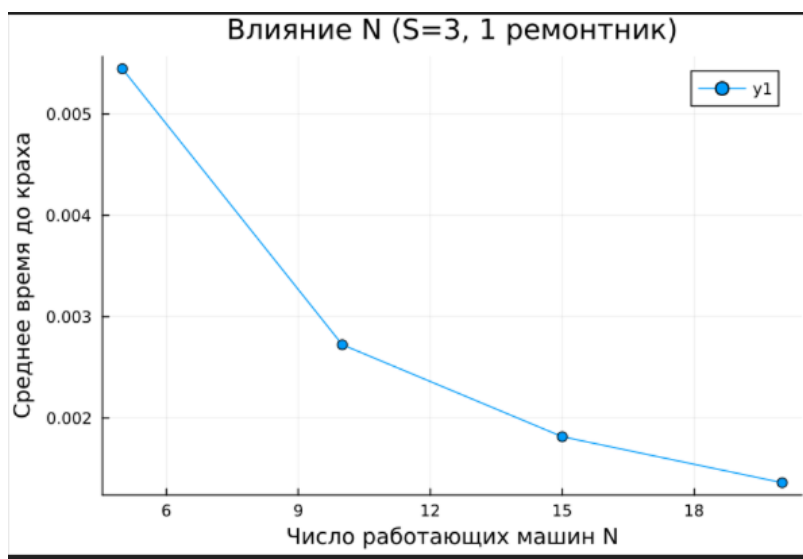


Рисунок 3.16: график 2

График влияние S (рис. 3.17).

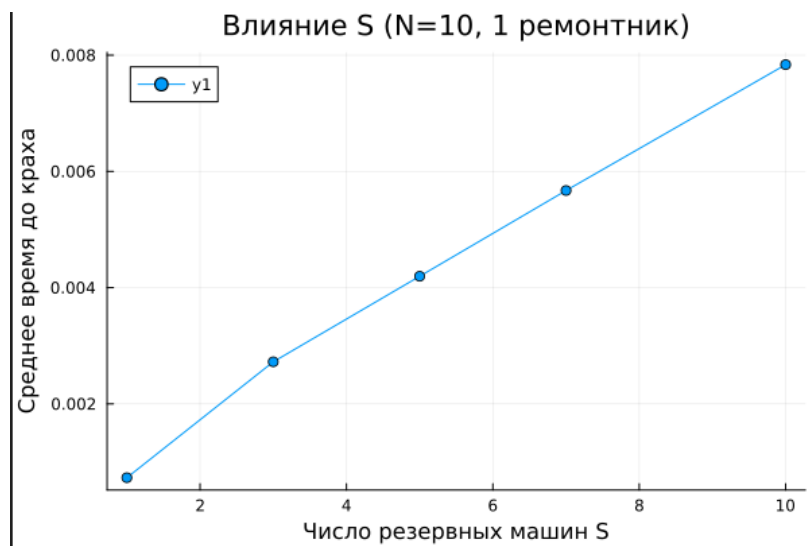


Рисунок 3.17: график 3

4 Выводы

Разработаны две рабочие модели СМО. Для М/М/с получены аналитические формулы (формулы Эрланга, Литтла) и построены графики зависимости P_{wait} и L_q от загрузки системы при разном числе каналов. Для модели Росса реализована событийная симуляция, исследовано влияние числа ремонтников, количества работающих машин N и резервных машин S на среднее время до краха. Построены графики, показывающие, что увеличение числа ремонтников и объёма резерва существенно повышает надёжность системы. Обе модели демонстрируют типичное для СМО поведение и могут быть использованы для анализа реальных систем массового обслуживания.